

PROJEKTFAHRPLAN

KI-Beschaffungsagent — Schaufler Fischer Group

Prototyp in 2 Wochen · Vollversion in 10 Wochen

Stand: Februar 2026 · Person A: Entwicklung · Person B: Frontend & Kommunikation

1. Projektziele & Rahmenbedingungen

Das Projekt wird modular gebaut. Jede externe Schnittstelle (ELO, Proleis, Outlook) ist austauschbar. Im Prototyp werden ELO und Proleis durch einfache Mocks/Fallbacks ersetzt — die echten Adapter kommen in Phase 2.

Phase	Ziel	Frist / Zahlung
Prototyp	AB-Workflow mit Mock-Daten + Dashboard	4 Wochen (Ziel: 2 Wochen) → 50% Zahlung
Vollversion	Alle Lastenheft-Anforderungen inkl. echter Proleis- & ELO-Anbindung	10 Wochen → restliche 50% Zahlung

2. Modulare Architektur — Adapter-Pattern

Jede externe Schnittstelle bekommt einen eigenen Adapter mit fixer interner Schnittstelle. Person A entwickelt immer gegen das Interface — nicht gegen das externe System direkt. Dadurch könnt ihr Proleis, ELO oder Outlook jederzeit austauschen ohne den Core anzufassen.

Modul	Interface (fix)	Prototyp (Woche 1-2)	Vollversion (Woche 3+)
E-Mail	get_new_mails() send_mail()	mock_adapter.py	graph_adapter.py
ERP / Bestelldaten	get_order() update_delivery_date()	mock_adapter.py	proleis_adapter.py
DMS / Ablage	store_document() tag_document()	filesystem_adapter.py	elo_adapter.py
KI-Extraktion	extract_attributes(doc)	Claude API	Claude API

Projektstruktur:

```
/agent  /core  orchestrator.py      ← Hauptlogik, kennt keine externen Systeme
ab_processor.py      ← KI-Extraktion & Abgleichslogik      escalation_engine.py ←
Eskalations- & Erinnerungslogik      scheduler.py          ← stündliches Polling
/adapters  /email  → base.py, graph_adapter.py, mock_adapter.py  /erp      →
base.py, proleis_adapter.py, mock_adapter.py      /dms      → base.py, elo_adapter.py,
filesystem_adapter.py  /dashboard      ← Person B (Streamlit)  config.yaml
← hier Adapter umschalten
```

config.yaml Beispiel:

```
# Prototyp email: mock erp: mock dms: filesystem # Vollversion # email: graph # erp:
proleis # dms: elo
```

3. Rollenaufteilung

Bereich	Person A — Entwicklung	Person B — Frontend & Kommunikation
Agent Core	orchestrator.py, ab_processor.py, escalation_engine.py, scheduler.py	—
Adapter	Alle Adapter bauen (Mock zuerst, echte später)	—
KI-Extraktion	Claude API Integration, Prompt Engineering für AB-Dokumente	Testfälle & Beispiel-ABs sammeln und aufbereiten
Dashboard	API-Endpunkte bereitstellen (FastAPI) damit Dashboard Daten hat	Streamlit App: Agent-Status, AB-Übersicht, Eskalationen, Tasks
Kommunikation	Technische Fragen an Schaufler IT klären	Projekt-Updates, Demo vorbereiten, Schaufler-Kontakt managen
Tests	Unit Tests für Core-Logik und Adapter	End-to-End Tests, Demo-Script, Abnahmeprüfung

4. Detaillierter Umsetzungsplan — Prototyp (2 Wochen)

Sprint 0 — Tag 1 (beide zusammen, ~4 Stunden)

Ziel: Alles was den Entwicklungsstart blockiert aus dem Weg räumen.

Aufgabe	Wer	Deadline
GitHub Repo erstellen, Ordnerstruktur anlegen, .env Template, README	Person A	Heute
config.yaml aufsetzen, Adapter-Interfaces (base.py) für alle 3 Module definieren	Person A	Heute
Schaufler IT kontaktieren: Outlook-Zugang (Graph API oder IMAP?), ELO-API-Version, Proleis-Zeitplan	Person B	Heute
3-5 echte Auftragsbestätigungen (PDF + E-Mail) vom Pilotlieferant anfordern	Person B	Heute
Standup-Zeit festlegen (täglich 15 Min), Kommunikationskanal klären	Beide	Heute

Woche 1 (Tag 2–7) — Agent Core mit Mock-Daten

⚠️ Kein Proleis, kein ELO in dieser Woche. Person A entwickelt komplett gegen Mock-Adapter. Person B baut das Dashboard parallel.

T a	Wer	Aufgabe	Ergebnis	Prio
-----	-----	---------	----------	------

g				
2	Person A	mock_email_adapter.py: simuliert eingehende E-Mails mit echten AB-Beispielen als .eml / JSON-Fixtures	E-Mails lesbar ohne Outlook	P1
2	Person A	mock_erp_adapter.py: Bestelldaten als JSON-Fixtures, get_order() / update_delivery_date() implementieren	ERP-Daten verfügbar	P1
2	Person B	Dashboard-Grundgerüst in Streamlit: Seitenstruktur, Navigation, Dummy-Daten einbinden	App läuft lokal	P1
3	Person A	KI-Extraktion (ab_processor.py): Claude API liest PDF/E-Mail-Text, extrahiert Liefertermin, Menge, Preis, Incoterm, Zahlungsbedingungen	Extraktion auf 5 Beispiel-ABs getestet	P1
3	Person B	AB-Übersichtstabelle im Dashboard: offene ABs, Status-Spalte, Liefertermin, Lieferant	Tabelle sichtbar mit Mock-Daten	P1
4	Person A	Abgleichslogik (ab_processor.py): extrahierte AB-Daten vs. ERP-Mock, Toleranzlogik gem. Lastenheft 4.0 (>5 Tage, jede Preis-/Mengenabw.)	Abweichungen werden erkannt	P1
4	Person A	mock_email_send: Eskalations-E-Mail wird generiert und in Konsole / Logfile ausgegeben (kein echtes Senden nötig)	E-Mail-Text sichtbar	P1
4	Person B	Eskalations-Ansicht im Dashboard: Liste der Abweichungen, Ampellogik (Grün/Gelb/Rot), generierte E-Mail-Vorschau	Ampellogik sichtbar	P1
5	Person A	Erinnerungslogik (escalation_engine.py): 3-stufig für fehlende ABs (nett → bestimmt mit CC Einkäufer → Teamleiter CC)	3 Erinnerungsstufen funktionieren	P1
5	Person A	filesystem_dms_adapter.py: Dokumente in lokalem Ordner ablegen + Metadaten-JSON speichern (Fallback für ELO)	Dokumente werden abgelegt	P1
5	Person B	Task-/To-do-Bereich im Dashboard: offene Aufgaben aus Eskalationen, Status änderbar (offen / in Klärung / erledigt)	Tasks verwaltbar	P2
6	Beide	End-to-End Test: kompletter Flow mit Mock-Daten (AB eingehend → Extraktion → Abgleich → Eskalation → Ablage → Dashboard)	Gesamtflow läuft durch	P1
7	Beide	Bugfixing, Edge Cases (kein Liefertermin im Dokument, falsches Format, doppelte AB), Logging verbessern	Agent ist stabil	P1

Woche 2 (Tag 8–14) — Outlook-Anbindung, Scheduler & Demo

⚠ ELO und Proleis bleiben weiter auf Mock/Filesystem. In dieser Woche kommt echter Outlook-Zugang dazu — falls Schaufler IT die Zugangsdaten bis dann geliefert hat. Sonst läuft der Prototyp weiter mit Mock-E-Mails.

T a g	Wer	Aufgabe	Ergebnis	Prio
8	Person A	graph_email_adapter.py oder imap_adapter.py: echter Outlook-Zugang, Postfach Order@schaufler.de anbinden	Echte E-Mails lesbar	P1*
8	Person B	Dashboard: Agent starten/stoppen Button, Live-Logs Ansicht, letzter Lauf-Timestamp	Steuerung im Dashboard	P1
9	Person A	scheduler.py: APScheduler einrichten, stündliches Polling, Erinnerungsfristen automatisch einhalten	Agent läuft vollautomatisch	P1
9	Person B	Dashboard polieren: Schaufler-Branding (Farben, Logo), mobile-freundliche Darstellung, Übersichts-KPIs (Anzahl offene ABs, Eskalationen)	Professionelles Aussehen	P2
10	Beide	Vollständiger Regressionstest: alle Szenarien (AB korrekt, AB abweichend, AB fehlt, doppelte AB)	Alle Szenarien getestet	P1
11	Person A	Docker-Setup: Agent + Dashboard in docker-compose, einfacher Start auf jedem Rechner	1-Befehl Start	P2
12	Person B	Demo vorbereiten: Testdaten aufbereiten, Demo-Script schreiben (Schritt für Schritt was gezeigt wird), Präsentation üben	Demo-Script fertig	P1
13	Beide	Puffer: letzte Fixes, README & kurze Doku für Übergabe, Demo nochmal durchspielen	Puffer	P2
14	Beide	PROTOTYP-DEMO bei Schaufler Fischer Group	★ Abnahme + 50% Zahlung	DEMO

* P1* = nur falls Schaufler IT bis Tag 8 Outlook-Zugangsdaten geliefert hat. Sonst weiter mit Mock — kein Blocker für die Demo.

5. Abnahmekriterien Prototyp

Der Prototyp gilt als abgenommen wenn alle P1-Kriterien erfüllt sind:

#	Prio	Kriterium	Status
1	P1	Agent liest eingehende ABs automatisch (echt via Outlook oder simuliert via Mock)	Offen

2	P1	AB (PDF) wird korrekt verarbeitet: alle 6 Attribute extrahiert (Liefertermin, Position, Menge, Preis, Incoterm, Zahlungsbedingungen)	Offen
3	P1	Abgleich mit ERP-Daten: Abweichungen korrekt erkannt (>5 Tage, jede Preis-/Mengenabweichung)	Offen
4	P1	Eskalations-E-Mail wird automatisch generiert (an Einkäufer, mit korrektem Inhalt)	Offen
5	P1	Fehlende AB: 3-stufige Erinnerungslogik läuft (nett → CC Einkäufer → CC Teamleiter)	Offen
6	P1	Dokument wird abgelegt (Filesystem-Fallback) und ist dem richtigen Vorgang zugeordnet	Offen
7	P1	Dashboard: Agent-Status, AB-Übersicht, Eskalationen, Agent starten/stoppen	Offen
8	P2	Docker: Agent + Dashboard starten mit einem Befehl	Offen
9	P2	Echter Outlook-Zugang verbunden (falls IT-Zugangsdaten rechtzeitig vorliegen)	Optional

6. Risiken & Gegenmassnahmen

Risiko	Wahrsch.	Gegenmassnahme
Outlook-Zugangsdaten kommen zu spät	MITTEL	Mock-Adapter läuft weiter — Demo funktioniert trotzdem vollständig
KI-Extraktion bei unbekanntem AB-Format unzuverlässig	MITTEL	Mehrere Beispiel-ABs vom Pilotlieferant vorab anfordern; Confidence-Score einbauen
Proleis-Schema nach Wechsel unbekannt	HOCH	Adapter-Pattern: Core-Code bleibt unberührt; Proleis-Adapter wird in Phase 2 gebaut
ELO-API komplex oder nicht verfügbar	HOCH	filesystem_adapter.py als vollwertiger Fallback für Prototyp; ELO kommt in Phase 2
Scope-Creep: Kunde will mehr im Prototyp	MITTEL	Schriftlichen Prototyp-Scope bestätigen lassen; Backlog für Phase 2 pflegen

7. Sofortmassnahmen — Was heute noch tun?

#	Aufgabe	Wer	Wann
1	GitHub Repo erstellen + Ordnerstruktur + Adapter-Interfaces anlegen	Person A	Heute
2	E-Mail an Schaufler IT: Outlook-Zugang (Graph API oder IMAP?), ELO-API-Version, Proleis Go-Live-Datum & Schema	Person B	Heute
3	3-5 echte Auftragsbestätigungen vom Pilotlieferant anfordern (PDF + E-Mail-Format)	Person B	Heute

4	Prototyp-Scope schriftlich an Schaufler bestätigen lassen (AB-Workflow + Dashboard, kein ELO/Proleis)	Person B	Morgen
5	Ersten Claude API-Key testen: AB-Beispiel-PDF an API schicken, Extraktion testen	Person A	Morgen
6	Tägliches Standup einrichten (15 Min, feste Uhrzeit)	Beide	Heute

Modular bauen, täglich kommunizieren, Scope halten — ELO & Proleis kommen in Phase 2.